



university of
 groningen

faculty of science
and engineering

Ethical Hacking - Assignment 2 - Group 8

Authors: Ioannis Loukas (s6378552) & Delia-Andreea Popa
(s5155606)

Contents

1	Challenge Design	2
2	Step-by-step Solution	3
3	Hints	5
4	Additional Tools	5

1 Challenge Design

The challenge is designed as a web-based exploitation scenario running inside a single Docker container. Attackers must discover, enumerate and compromise an intentionally vulnerable Django web application, access an exposed SQLite database and retrieve the flag stored there. As such, the created challenge involves multiple stages: service discovery (e.g., port scanning), login credential brute-forcing, insecure data exposure.

The vulnerable Django web application exposes three routes:

- / - default landing page, leads user to login page
- /login/ - authentication page
- /dashboard/ - administrator "protected" page accessible only after a valid administrator role login

On the landing page, the button that leads the user to the login page is hidden within the HTML elements depicted in the page. The user has to find said button in order to proceed to the login page.

For authentication, we used the built-in Django authentication system and automatically ran a script during image build time to create a weak administrator user (password was taken from `500-worst-passwords.txt`). Moreover, to accept credential brute-force attempts, we disabled CSRF protection for the login page. We made the username field uneditable, indicating that the only valid username that can be used further is the one displayed on the page (in our case the username is admin).

To further "guide" the attacker into retrieving the hidden flag and upon successful authentication with the found administrator credentials, the dashboard page showcases a message indicating that "valuable information" is stored in the SQLite database included in the web application. This is the location where the flag is hidden.

2 Step-by-step Solution

The solution to the designed challenge involves the following steps:

Step 1: The attacker scans the target with the `nmap` command to check for open/filtered ports (the command we used is `nmap -p- 10.10.0.0/24`). Upon successful use of this command, the attacker should find `8000/tcp filtered http-alt` for host `10.10.0.230`.

Step 2: The attacker should try to access the web application listening on port 8000 by running a `curl` command (the command we used is `curl http://localhost:8000/`). Upon successful use of this command, the attacker should be prompted with the `index.html` code, where a button to access the login page is hidden within the many HTML elements that the page displays. The button can be either found by inspecting the actual HTML code or by simply opening the page in a browser and hovering over the page to find the button. By following the first approach, the attacker can find the URL corresponding to the login page (and thus try a `curl` command again to access it, the command we used being `curl http://localhost:8000/login/`) and by following the second approach, the attacker can simply click the button and be prompted with the login page.

Step 3: The attacker is now prompted with the `login.html` code or the actual login page loaded in the browser. Here, the attacker is prompted with the login logic: the username is added to the page as a read only value (username: admin), so the attacker has to guess the password to proceed further. Inside the login page, there is a hint leading to which command should be used by the attacker in order to guess the password (use the `hydra` command). Thus, the attacker tries to brute-force the login. We used the following command:

```
hydra -l admin -P 500-worst-passwords.txt -s 8000
localhost http-post-form
"/login/:username=~USER~&password=~PASS~:
credentials are invalid"
```

Upon successful finding of the password, the attacker logs in (username=admin, password=iceman). The attacker can either log in through the page displayed

in the browser or through the terminal by using the following command: `curl -c session_cookie.txt -d "username=admin&password=iceman" -X POST -i http://localhost:8000/login/`). If the attacker does not log in, they will not have access to the dashboard page, since it is protected by a "login_required" tag in the source code.

Step 4: If the attacker logged in through the page displayed in the browser, they are prompted with the `dashboard.html` page. If the attacker logged in by using the aforementioned `curl` command, the attacker has to run the following command in order to gain access to the HTML code of the dashboard page: `curl -b session_cookie.txt -L http://localhost:8000/dashboard/`. Displayed on this page is a hint leading to the web application's SQLite database, which the attacker has to download somehow.

Step 5: The attacker can either follow the hint on the dashboard page and guess the URL in the browser page which will automatically download the SQLite database file, or they can use a `wget` command in the terminal to download it. The command we used is: `wget http://localhost:8000/static/db.sqlite`.

Step 6: Upon successful download of the SQLite database file, the attacker has to inspect its content. The commands we used to inspect the database are the following:

```
$ sqlite3 db.sqlite
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .tables
secrets
sqlite> SELECT * from secrets;
1|EH2025{33d4fc1c0d28b80a9a3b93f5c2523103fc06520374a231ca55bcf825aa4579f9}
```

By running the above commands, the flag is retrieved.

3 Hints

The following hints have been used in the design of this challenge:

- Hint 1: Text saying "There must be a login button somewhere..." added on top of the landing page, as well as a `div` with a button hidden in the middle of the page.
- Hint 2: Password placeholder changed in the login page to link to video of Marvel's Avengers titled "Avengers vs HYDRA [...]" and text appended saying "Avengers vs WHO???" to indicate the need to use the `hydra` command.
- Hint 3: Message displayed in the dashboard page - "Very important information stored in URL: `static/db.sqlite`".

4 Additional Tools

Additional tools required to be installed in order to perform the designed challenge would be the following:

- `hydra` - command: `sudo apt install hydra`
- `sqlite3` - command `sudo apt install sqlite3`